

Function Vectors in Vision Language Models

Princeton Liu, Shuhao Fu, Hongjing Lu

Computational Vision and Learning Lab

January 13, 2026

- What are "function vectors"?
- Are they applicable to "vision language models"?

Introduction to Language Models

- Simplified idea: A Language Model (LM) inputs text ("context") and predicts the next word ("prediction")
- Example:
 - Context: "What is the capital of France?"
 - Prediction: "Paris."
- In-context learning (ICL): an emergent property of LM's where they perform a new task based only on context
- Example:
 - Context: "Old : New , Short : Tall , Fast : "
 - Prediction: "Slow"

Introduction to Function Vectors

- A Function Vector (FV) is a vector representation of an ICL task (antonym, english-french translation, etc.)
- You can "inject" Function Vectors into LMs to do ICL tasks WITHOUT sufficient context
 - ① Context: "Complex : "
 - ② Inject Antonym Function Vector into the model's hidden state
 - ③ Prediction: "Simple."
- Normally the only way to "influence" an LM's response is through giving it richer context (no guarantees)
- With function vectors, we can directly influence an LM's behavior without context.

Vision-Language Models

- A vision language model (such as CLIP) inputs two things: an image (pixels) and text (string)
- It transforms the image into a vector and transform the text into a different vector ("embeddings").
- We use CLIP ViT B-16 (CLIP) model for our experiments
- CLIP is trained to "align" these two vectors (point in the same direction) so that it connects the image space and text space.
- Unlike LMs, CLIP is NOT generative (it doesn't generate words/images), so it does NOT have ICL capabilities.

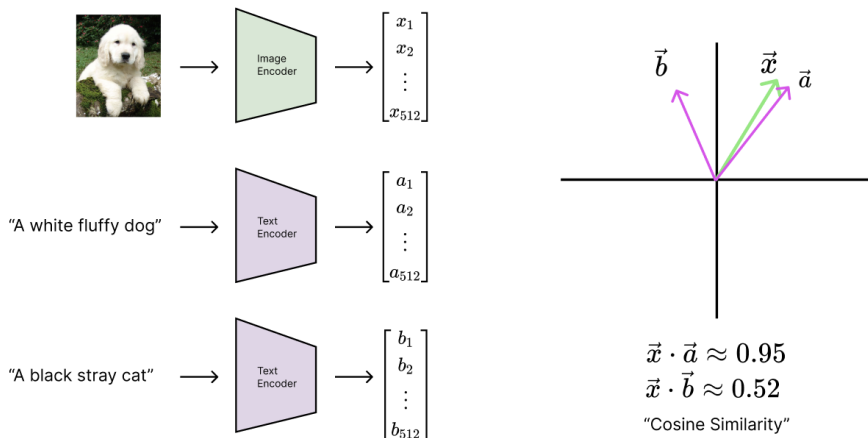


Figure 1: Text/Image embeddings

Image Encoder

- To encode the whole image, the model encodes sections (patches) of the image individually and then "mixes" the vectors through "attention"
- There is also an important embedding called the "CLS token" (classification token) which acts as a summary of the entire image.
- The final embedding is a concatenation of the CLS embedding and the patch embeddings.

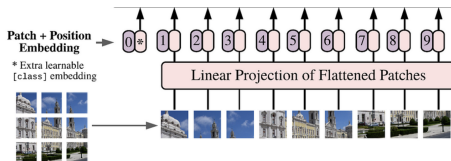


Figure 2: CLS and Patch embeddings. Adapted from Dosovitskiy et al. (2021).

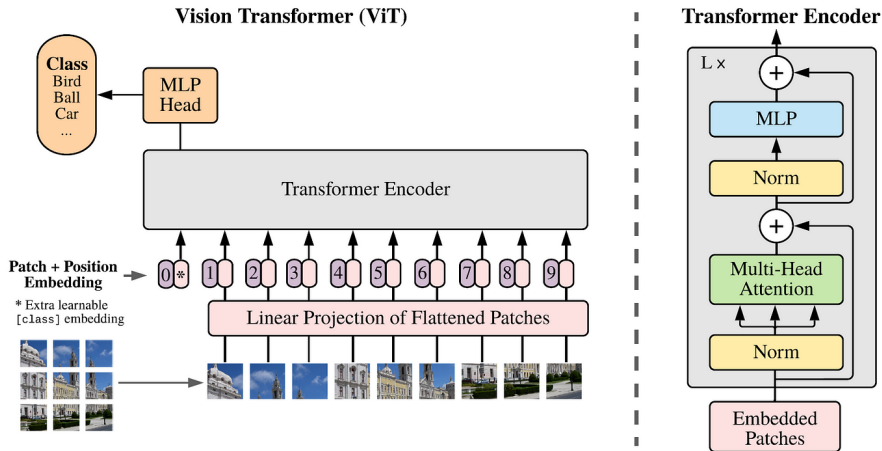


Figure 3: ViT-B/16 Transformer encoder architecture. Adapted from Dosovitskiy et al. (2021).

- CLIP ViT B-16 is very good at determining the subject of an image.

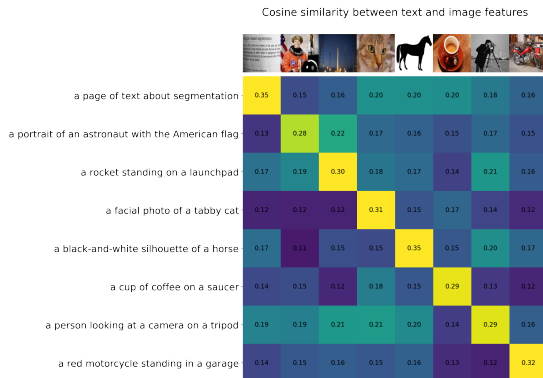


Figure 4: CLIP General Image Cosine Similarity

- A limitation of CLIP ViT B-16 is its lack of understanding "relational reasoning."

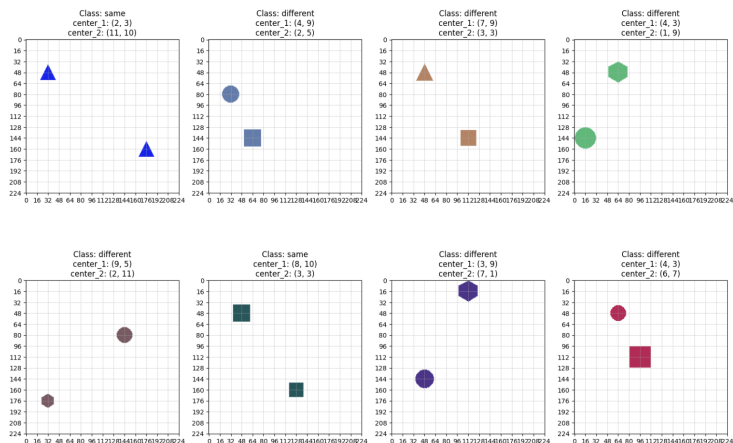


Figure 5: Same-Different relation dataset

- A limitation of CLIP ViT B-16 is its lack of understanding "relational reasoning."

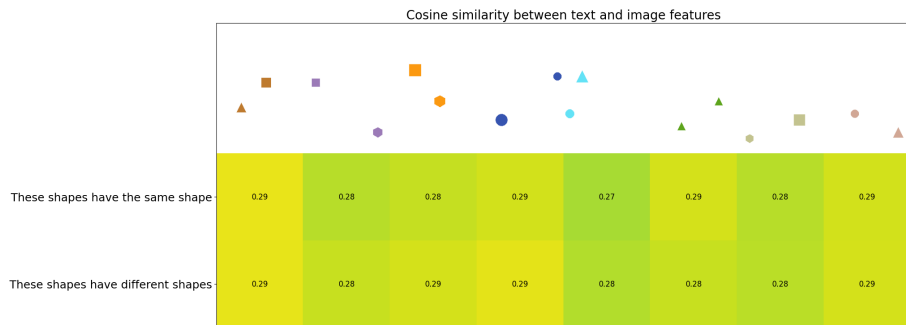


Figure 6: CLIP Same-Different Cosine Similarity

Fine-Tuning

- Goal: enable CLIP to have relational reasoning capabilities
- Naive solution: fine-tune the model (change model parameters) on a relational reasoning dataset
- Issue: Changing the model parameters will change the model's behavior (and the entire embedding space) in general

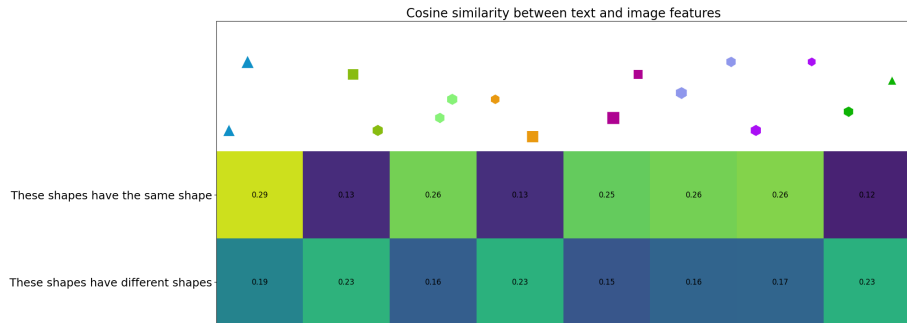


Figure 7: Fine-tuned CLIP Relational Reasoning Cosine Similarity

Fine-Tuning

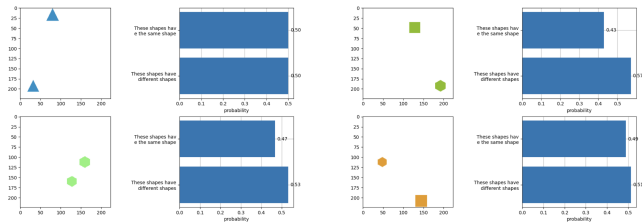


Figure 8: Non-fine-tuned CLIP same-different probabilities.

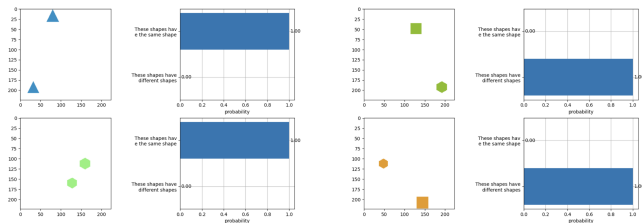


Figure 9: Fine-tuned CLIP same-different probabilities.

Average CLS Activations

- New Goal: enable CLIP to have relational reasoning capabilities without changing parameters
- Average CLS Activations solution: for a relational reasoning task, take the average CLS embedding (AKA activation) for a fine-tuned model.
- Intuition: CLS activation is a "summary" of the image, so the average CLS activation over many images which contain a relation will contain that relation.

- Method:
 - ① Create a dataset for a relational reasoning task
 - ② Fine-tune CLIP on that dataset
 - ③ Calculate average CLS activation for fine-tuned CLIP
 - ④ Find the accuracy of a non-fine-tuned CLIP model with injected average activations into it

Average CLS Activations

- The encoder has 12 layers, so we can choose which layers we want to inject the embeddings into.

(Non-fine-tuned model)

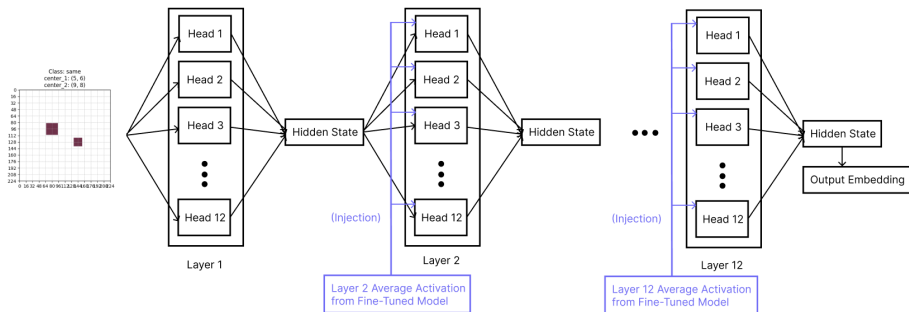


Figure 10: Example of injection.

Average CLS Activations

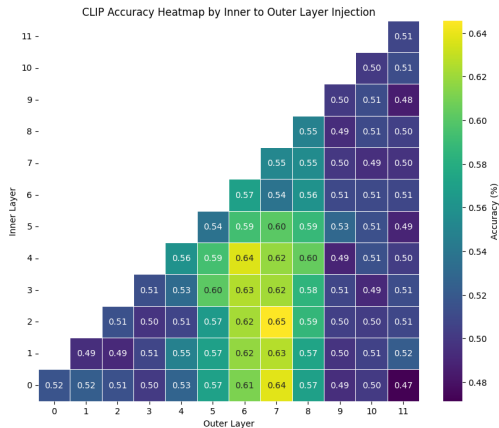


Figure 11: Accuracy based on which layers are injected into

Average Indirect Effect

- Can we get better performance by constructing Function Vectors?
- Each layer has 12 heads. Our goal is to find out which of the 144 heads contribute most to relational reasoning.
- We find the "average indirect effect" for each head, which measures how much an injection of that specific head improves performance.



Figure 12: Average Indirect Effect Heatmap.

Function Vector Injection

- Only inject into the heads which are involved in the reasoning "circuit"

(Non-fine-tuned model)

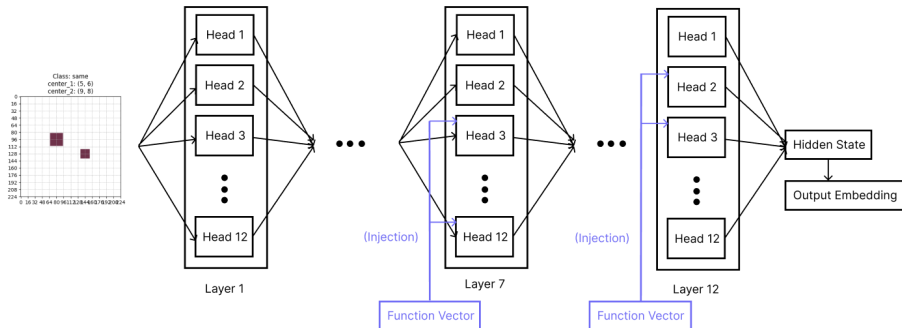


Figure 13: Example of function vector injection.

- Given the AIE for each head, what kind of methods do we do to construct a CLIP function vector?
- Traditionally, function vectors are constructed by summing entire head activations.
- Instead of only injecting into the CLS embeddings, we can try injecting into the patch embeddings as well.